

HTTP in Detail

Learning Notes — Requests, Responses, Methods, Status Codes, Headers & Cookies

"HTTP is the foundation of data communication on the World Wide Web — every webpage, API call, and web application you use relies on it."

 HTTP & HTTPS	 Requests	 Responses	 Status Codes	 Headers & Cookies
--	--	---	--	---

1. What Is HTTP & HTTPS?

HTTP (HyperText Transfer Protocol) is the set of rules used for communicating with web servers to send and receive web content — HTML pages, images, videos, JSON data, and more. It was developed by Tim Berners-Lee between 1989 and 1991 alongside the World Wide Web. Every time you load a website, your browser is making HTTP requests and receiving HTTP responses.

Aspect	HTTP	HTTPS
Full name	HyperText Transfer Protocol	HyperText Transfer Protocol Secure
Encryption	None — data transmitted in plain text	TLS/SSL encryption — data is encrypted in transit
Port	Port 80 (default)	Port 443 (default)
Certificate	No certificate required	Requires a TLS certificate from a Certificate Authority
Data privacy	Anyone on the network can read the data	Data is unreadable to interceptors — only client & server can decrypt
Identity	No server authentication — could be impersonated	Certificate verifies you are talking to the correct server
URL prefix	http://	https://
Use today	Largely deprecated — insecure for any sensitive data	Standard for all modern websites — browsers warn on HTTP

Security Note — URL Query Strings

Query string parameters (`?id=1`) are sent in plain text and are visible in server logs, browser history, and to anyone monitoring traffic. Never send sensitive data (passwords, tokens) in query strings. Query strings are also the primary vector for SQL injection and parameter manipulation attacks — always validate and sanitise all query parameters server-side.

3. HTTP Requests ---

An HTTP request is a message sent by the client (typically a web browser) to a web server asking for an action to be performed. Every request follows a defined structure and contains specific components that tell the server what is needed and how to respond.

3.1 HTTP Request Structure — Annotated

```
GET / HTTP/1.1
# ^ ^ ^
# | |   HTTP version (1.1 most common; HTTP/2 and HTTP/3 also in use)
# |     Path being requested (/ = homepage)
#       HTTP Method (GET, POST, PUT, DELETE...)

Host: tryhackme.com
#   Which website to load (critical for servers hosting multiple sites)

User-Agent: Mozilla/5.0 Firefox/87.0
#   Identifies the browser and version making the request

Referer: https://tryhackme.com/
#   Which page sent the user to this URL

Accept-Encoding: gzip, deflate, br
#   Compression formats the browser supports

Cookie: theme=dark; session=abc123xyz
#   Stored cookie data sent with every request to this server

# [blank line signals end of headers]
# [optional request body follows — present in POST/PUT]
```

3.2 HTTP Response Structure — Annotated

```
HTTP/1.1 200 OK
# ^      ^      ^
# |      |      | — Status text (human-readable description)
# |      |      | — Status code (machine-readable result)
# |      |      | — HTTP version the server is using

Server: nginx/1.19.0
# — Web server software and version (useful for recon!)

Date: Fri, 09 Apr 2021 13:34:03 GMT
# — Server's current date, time and timezone

Content-Type: text/html; charset=utf-8
# — Type of data in the response body (HTML, JSON, image, etc.)

Content-Length: 98
# — Number of bytes in the response body

# [blank line signals end of headers — start of body]

<html>
<head><title>TryHackMe</title></head>
<body>Welcome To TryHackMe.com</body>
</html>
```

The Blank Line Is Required

Both HTTP requests and responses use a blank line to separate the headers section from the body. The server knows the headers have ended when it encounters a blank line (CRLF CRLF). This is a fixed part of the HTTP protocol structure — missing it would cause parsing failures.

4. HTTP Methods

HTTP methods (also called HTTP verbs) tell the web server what action the client wants to perform. While there are many HTTP methods, the four most commonly encountered in web development and security work are GET, POST, PUT, and DELETE — collectively representing the four CRUD operations.

CRUD Operations

HTTP methods map directly to CRUD — the four fundamental operations of persistent data management: Create (POST), Read (GET), Update (PUT/PATCH), Delete (DELETE). RESTful APIs are designed around this mapping, making HTTP method knowledge essential for both web development and web application security testing.

GET — Read — Retrieve data from the server

Requests data from a web server. Used for loading web pages, fetching images, retrieving API data. Parameters are passed in the URL query string (?key=value). Safe and idempotent — calling GET multiple times does not change server state.

Example: `GET /blog?id=1 HTTP/1.1`

Request body required: No — GET requests should not have a request body

Security note: GET parameters are visible in URLs, browser history, server logs, and Referer headers. Never use GET for sensitive data (passwords, tokens). SQL injection can be performed via GET parameters.

POST — Create — Submit data to the server

Sends data to the web server, typically to create new records. Data is sent in the request body — not visible in the URL. Used for login forms, sign-up forms, submitting comments, uploading files. Not idempotent — calling POST multiple times may create duplicate records.

Example: `POST /login HTTP/1.1 [body: username=thm&password=letmein]`

Request body required: Yes — POST data is sent in the request body

Security note: POST body data is not visible in URLs but IS visible to anyone who can intercept traffic (use HTTPS). POST is the primary vector for many attacks: SQL injection via form fields, CSRF (Cross-Site Request Forgery), and file upload vulnerabilities.

PUT — Update — Replace/update existing data

Sends data to the web server to update an existing resource. Replaces the entire resource at the specified URL with the new data provided. Idempotent — calling PUT multiple times with the same data produces the same result.

Example: `PUT /user/2 HTTP/1.1 [body: username=admin]`

Request body required: Yes — PUT sends the updated data in the request body

Security note: Unauthenticated or improperly authorised PUT endpoints allow attackers to overwrite user data, escalate privileges, or deface content. The example in the room updates a username to 'admin' — demonstrating insecure direct object reference (IDOR) risk.

DELETE — Delete — Remove data from the server

Requests that the web server deletes the specified resource. Idempotent — deleting a resource twice has the same result (it is gone). Used in REST APIs for removing records, files, or user accounts.

Example: `DELETE /user/1 HTTP/1.1`

Request body required: Optional — may include body specifying what to delete

Security note: Unauthenticated DELETE endpoints are catastrophic — attackers can delete users, records, or files. Always require authentication and authorisation checks before processing DELETE requests. Mass account deletion is a common attack against vulnerable APIs.

4.1 Additional HTTP Methods

Method	Purpose
PATCH	Partial update — modifies only specified fields of a resource (unlike PUT which replaces the whole resource).
HEAD	Identical to GET but returns only headers, no body. Used to check if a resource exists and inspect its metadata without downloading it.
OPTIONS	Returns the HTTP methods supported by the server for a specific resource. Used in CORS preflight requests by browsers.
TRACE	Diagnostic method — performs a loop-back test, echoing the received request back to the client. Often disabled as it can reveal proxy headers and enable cross-site tracing (XST) attacks.

5. HTTP Status Codes

HTTP status codes are three-digit numbers returned by the server in every response, indicating the outcome of the request. They are grouped into five classes by their first digit, each representing a different category of outcome.

1xx Informational	2xx Success	3xx Redirect	4xx Client Error	5xx Server Error
-----------------------------	-----------------------	------------------------	----------------------------	----------------------------

Code	Name	When You See It
1xx	Informational	Sent to tell the client the first part of their request has been accepted and they should continue sending the rest.
100	Continue	Server has received the request headers; client should proceed to send the body.
2xx	Success	Request was received and processed successfully.

200	OK	Request completed successfully. Standard response for successful GET requests.
201	Created	A new resource has been created — e.g. new user registered or new blog post published. Answers: 'What code for a new user/post?' → 201
3xx	Redirect	Client is being redirected to a different resource.
301	Moved Permanently	Resource has moved to a new URL permanently. Browsers update bookmarks. SEO transfers to new URL.
302	Found (Moved Temporarily)	Resource temporarily at a different URL. Browser redirects but retains original URL.
4xx	Client Error	The problem is with the request the client sent.
400	Bad Request	Server could not understand the request — malformed syntax, invalid parameters, missing required fields.
401	Not Authorised	Authentication required. You must log in before accessing this resource. Answers: 'Code if editing profile without logging in?' → 401
403	Forbidden	Authenticated but not permitted — you lack the required permissions for this resource.
404	Page Not Found	The requested resource does not exist on the server. The most recognised error code on the internet.
405	Method Not Allowed	The HTTP method used is not supported for this endpoint. e.g. sending GET to an endpoint expecting POST.
5xx	Server Error	The problem is with the server — it failed to handle a valid request.
500	Internal Server Error	Generic server crash — something went wrong but the server doesn't know exactly what. Often reveals stack traces in error messages.
503	Service Unavailable	Server cannot handle the request — either overloaded or down for maintenance. Answers: 'Code if server cannot access database?' → 503

6. HTTP Headers

Headers are additional pieces of metadata sent alongside both requests and responses. They provide context about the client, the server, the data being sent, and how it should be handled. While no single header is strictly required, most communication would fail or be suboptimal without them.

6.1 Common Request Headers — Client to Server

Header	Purpose
Host	Specifies which website to request from the server. Essential — one server can host multiple websites (virtual hosting). Without Host, the server returns its default site. Answer: 'Header telling server which website?' → Host
User-Agent	Identifies the browser software and version making the request. Helps servers serve browser-compatible content. Answer: 'Header telling server what browser is being used?' → User-Agent
Content-Length	When sending data (POST/PUT), tells the server how many bytes to expect in the request body. Prevents the server from waiting indefinitely for more data.
Accept-Encoding	Tells the server which compression formats (gzip, deflate, br) the client supports. Allows the server to compress responses for faster transmission.
Cookie	Sends stored cookie data back to the server with every request to that domain. Used for session management, authentication, and personalisation preferences.
Authorization	Carries credentials for authenticating to the server — typically 'Bearer <token>' for APIs or 'Basic <base64>' for basic auth.
Referer	The URL of the page that linked to the current request. Used by servers for analytics and access control.
Accept	Tells the server what content types the client can handle (e.g. text/html, application/json, image/webp).

6.2 Common Response Headers — Server to Client

Header	Purpose
Set-Cookie	Instructs the browser to store a cookie — sent back to the server on all subsequent requests. Answer: 'Header used to save cookies to your computer?' → Set-Cookie
Cache-Control	Instructs the browser how long to cache the response before making a fresh request. e.g. max-age=3600 means cache for 1 hour.
Content-Type	Tells the browser what type of data is in the response body — HTML, CSS, JavaScript, JSON, image/png, video/mp4, etc. Browser uses this to decide how to render the data. Answer: 'Header telling browser what type of data is returned?' → Content-Type
Content-Encoding	Tells the client what compression method was used on the response body (gzip, br) so it can decompress it.
Content-Length	Number of bytes in the response body. Confirms no data was lost in transit. Answer: 'Response header telling browser how much data to expect?' → Content-Length
Server	Identifies the web server software and version (e.g. nginx/1.19.0). Useful for recon — reveals technology stack.

Location	Used with 3xx redirect responses — tells the browser the new URL to navigate to.
Strict-Transport-Security	Instructs browsers to only communicate over HTTPS for this domain for a specified period. Prevents protocol downgrade attacks.

7. Cookies

HTTP is a stateless protocol — each request is treated as completely independent, with no memory of previous requests. Cookies solve this problem. They are small pieces of data stored by the browser and sent back to the server with every subsequent request to the same domain, allowing the server to maintain state across multiple requests.

Why Cookies Exist

Without cookies, you would need to log in again on every single page of a website. Cookies allow the server to recognise you across multiple requests by storing a session token in your browser. When you visit the site again, your browser sends the cookie, and the server recognises your session — you stay logged in.

7.1 How Cookies Work — The Full Flow

Step	Action	Detail
1	Browser sends request	You log in to a website: POST /login with username and password.
2	Server authenticates	Server validates credentials, creates a session record, generates a unique session token.
3	Server sets cookie	Response includes header: Set-Cookie: session=abc123xyz; HttpOnly; Secure; SameSite=Strict
4	Browser stores cookie	Browser saves the cookie (domain, name, value, flags) in its cookie store.
5	Browser sends cookie back	Every subsequent request to that domain includes: Cookie: session=abc123xyz
6	Server validates session	Server looks up the session token, recognises the user, serves the authenticated content.
7	Cookie expiry / logout	Cookie expires based on Max-Age/Expires attribute, or is deleted on logout via Set-Cookie with an expired date.

7.2 Cookie Security Attributes

Attribute	Effect & Security Impact
HttpOnly	Cookie cannot be read by JavaScript (document.cookie). Prevents XSS attacks from stealing session tokens. ALWAYS set on session cookies.
Secure	Cookie is only sent over HTTPS connections — never over plain HTTP. Prevents interception on unencrypted connections.
SameSite=Strict	Cookie is only sent on requests originating from the same site. Prevents CSRF (Cross-Site Request Forgery) attacks.
SameSite=Lax	Cookie is sent on same-site requests and top-level navigations (clicking a link). Partial CSRF protection with better usability.
SameSite=None	Cookie is sent on all cross-site requests (requires Secure). Used for legitimate third-party integrations but increases CSRF risk.
Expires / Max-Age	Sets when the cookie expires. Session cookies (no expiry) are deleted when the browser closes. Persistent cookies survive browser restarts.
Domain	Specifies which domains the cookie is sent to. If set to .example.com, cookie is sent to all subdomains.
Path	Restricts the cookie to a specific path within the domain. Cookie is only sent for requests to that path and below.

Cookie Manipulation Attack

The room demonstrates a cookie manipulation attack: if a server sets cookie name=Adam and does not validate the cookie value server-side, an attacker can change it to name=admin in their browser — instantly gaining admin access. This is why cookies must NEVER be trusted as authoritative data — always validate session state server-side.

8. Hands-On Lab — HTTP Request Simulator

The final task in the room is an interactive HTTP Request/Response Simulator. You send real HTTP requests to a mock web application and observe exactly how the request and response look. Each exercise reveals a flag on success.

#	Method + Path	Flag	What It Demonstrates
1	GET /room	THM{YOU'RE_IN_THE_ROOM}	Basic GET request — retrieving a page by path.
2	GET /blog?id=1	THM{YOU_FOUND_THE_BLOG}	GET with query string parameter (?id=1) to retrieve a specific resource.
3	DELETE /user/1	THM{USER_IS_DELETED}	DELETE request targeting a specific user by ID — shows destructive method power.

4	PUT /user/2?username=admin	THM{USER_HAS_UPDATED}	PUT with parameter to update a user record — demonstrates IDOR risk if unauthenticated.
5	POST /login [username=thm, password=letmein]	THM{HTTP_REQUEST_MASTER}	POST with body parameters — credential submission, the most common web form pattern.

All 5 Flags Captured

THM{YOU'RE_IN_THE_ROOM} • THM{YOU_FOUND_THE_BLOG} •
 THM{USER_IS_DELETED} • THM{USER_HAS_UPDATED} •
 THM{HTTP_REQUEST_MASTER} — Each flag reinforces a different HTTP method, confirming mastery of GET, POST, PUT, and DELETE requests in context.

9. Security Implications

Every HTTP Request Is an Attack Vector

Every component of an HTTP request — method, path, query parameters, headers, body — can be manipulated by an attacker. Web application security is fundamentally about validating and sanitising every piece of incoming data, regardless of where it arrives in the request.

HTTP Exposes Everything — HTTPS Is Non-Negotiable

Plain HTTP transmits all data — credentials, session tokens, personal information — as unencrypted text. Anyone on the same network (coffee shop Wi-Fi, corporate LAN, or a compromised router) can read everything. HTTPS with a valid TLS certificate is the baseline requirement for any modern web application.

Server Headers Reveal Technology Stack

The Server response header (e.g. Server: nginx/1.19.0) and X-Powered-By header (e.g. PHP/7.4.3) reveal the web server software and version. Penetration testers use this for recon — known CVEs for that exact version can then be researched. Best practice is to suppress or obscure these headers in production.

Status Codes Reveal Application Logic

The difference between a 401 (Not Authorised) and 403 (Forbidden) response reveals whether authentication is the issue (401 — you are not logged in) or authorisation (403 — you are logged in but not permitted). This helps attackers understand the application's access control model. 500 errors may expose stack traces and internal paths.

Cookie Theft = Account Takeover

Session cookies without HttpOnly can be stolen via XSS (Cross-Site Scripting) — a script injected into the page reads document.cookie and sends it to the attacker. The attacker replays the stolen cookie to impersonate the victim. HttpOnly, Secure, and SameSite attributes are the defences. Never store sensitive data (passwords, PII) in cookies.

CSRF — Cookies Are Sent Automatically

Because browsers automatically attach cookies to every request to a domain, a malicious website can trick a logged-in user's browser into sending authenticated requests to another site (CSRF — Cross-Site Request Forgery). SameSite=Strict/Lax cookie attributes and CSRF tokens in forms are the primary defences.

10. Key Takeaways & Glossary

10.1 Room Answers — Quick Reference

Question	Answer
What does HTTP stand for?	HyperText Transfer Protocol
What protocol does HTTPS use for encryption?	TLS/SSL
What HTTP protocol version is shown in the example?	HTTP/1.1
What response header tells the browser how much data?	Content-Length
What method would be used to create a new user account?	POST
What method would be used to remove an uploaded picture?	DELETE
What method would be used to view a news article?	GET
Response code if you've created a new user/post?	201
Response code if a page doesn't exist?	404
Response code if server can't access database / crashes?	503
Response code if editing profile without logging in?	401
What header tells the server what browser is being used?	User-Agent
What header tells the browser what type of data returned?	Content-Type
What header tells the web server which website is requested?	Host
Which header is used to save cookies to your computer?	Set-Cookie

10.2 Core Glossary

Term	Definition
HTTP	HyperText Transfer Protocol — rules for web client-server communication. Developed by Tim Berners-Lee 1989–1991.
HTTPS	HTTP Secure — HTTP with TLS encryption. Encrypts data and authenticates the server via certificate.
URL	Uniform Resource Locator — full address of a web resource. Components: scheme, user, password, host, port, path, query, fragment.
Query String	?key=value parameters appended to a URL. Visible in logs and browser history. Never use for sensitive data.
Fragment	#anchor — identifies a section within a page. NOT sent to the server — processed by the browser only.
GET	HTTP method — retrieves data. Parameters in URL. Safe and idempotent.
POST	HTTP method — submits data in body to create resources. Not idempotent.
PUT	HTTP method — updates an existing resource with new data. Idempotent.
DELETE	HTTP method — removes a resource. Idempotent.
Status Code	Three-digit server response code indicating outcome: 2xx success, 3xx redirect, 4xx client error, 5xx server error.
200 OK	Request completed successfully.
201 Created	New resource successfully created (new user, new post).
301 / 302	Permanent / Temporary redirect to a new URL.
401	Not Authorised — authentication required (not logged in).
403	Forbidden — authenticated but lacks permission.
404	Not Found — resource does not exist.
500	Internal Server Error — generic server-side crash.
503	Service Unavailable — server overloaded or in maintenance.
Host header	Request header identifying which website is being requested from a shared server.
User-Agent	Request header identifying the browser software and version.
Content-Type	Response header declaring the type of data in the body (HTML, JSON, image, etc.).
Content-Length	Response header declaring the byte size of the response body.
Set-Cookie	Response header instructing browser to store a cookie.
Cookie	Request header sending stored cookie data to the server.
HttpOnly	Cookie attribute preventing JavaScript access — blocks XSS cookie theft.

Secure	Cookie attribute — only sent over HTTPS connections.
SameSite	Cookie attribute controlling cross-site sending — prevents CSRF attacks.
Session token	A unique, random string stored in a cookie to identify an authenticated session.
Stateless	HTTP has no built-in memory — each request is independent. Cookies provide the state layer.

10.3 Next Steps on TryHackMe

- **Immediate:** Continue with How Websites Work — HTML, JavaScript, sensitive data exposure in source code.
- **Essential:** Complete Burp Suite: The Basics — intercept, inspect, and modify HTTP requests in real time.
- **Security:** Study the OWASP Top 10 room — the most critical web application security risks, all involving HTTP.
- **Path:** Take the Web Fundamentals learning path for a complete grounding in web security.
- **Advanced:** Learn SQL Injection, XSS, and CSRF in dedicated rooms — all exploit HTTP components covered here.

HTTP Is the Language of Web Security

Every web application vulnerability — SQL injection, XSS, CSRF, IDOR, authentication bypass, directory traversal — is exploited through HTTP. Understanding HTTP requests, responses, methods, status codes, headers, and cookies in detail is not optional background knowledge — it is the core skill that all web security work builds on.

Source: TryHackMe — HTTP in Detail Room | tryhackme.com/room/httpindetail